AI Edge (Computing)

5. Training a model for Edge AI

hogeschool vives

1

The "deep learning" workflow (or any other AI model)

The basic steps

1. Decide on a goal
2. Collect a dataset
3. Design a model architecture
4. Train the model
5. Convert the model
6. Run inference
7. Evaluate and troubleshoot

2

hogeschool vives

2

The "deep learning" workflow (or any other AI model)

The basic steps

1. Decide on a goal
2. Collect a dataset
3. Design a model architecture
4. Train the model
5. Convert the model
6. Run inference
7. Evaluate and troubleshoot

3

hogeschool vives

3

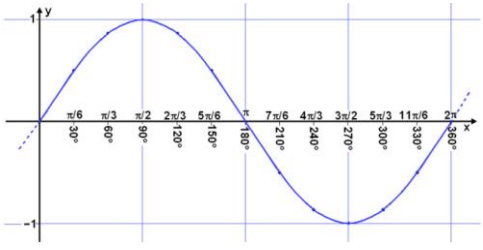
The "deep learning" workflow

The basic steps

1. Decide on a goal

We want to predict a sine wave: given an input value (x), predict the output value (sin(x))

→ What type of model is this?



4

hogeschool vives

4

The "deep learning" workflow

The basic steps

Sidenotes

- If you need to calculate a sine wave, please use a built-in sine function (math.h) rather than using deep learning
- If you don't have a built-in sine function at hand, you can construct a basic look-up table

5

hogeschool vives

5

The "deep learning" workflow

The basic steps

The required toolchains

- Python, Jupyter
- Google Colab
- TensorFlow & Keras

6

hogeschool vives

6

The "deep learning" workflow

The basic steps

The required libraries

```
# TensorFlow is an open source machine learning library
import tensorflow as tf
# Keras is TensorFlow's high-level API for deep learning
from tensorflow import keras
# Numpy is a math library
import numpy as np
# Pandas is a data manipulation library
import pandas as pd
# Matplotlib is a graphing library
import matplotlib.pyplot as plt
# Math is Python's math library
import math
# Time is a timing library
import time
```

7

hogeschool vives

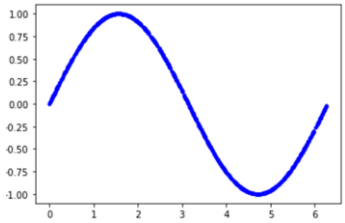
7

The "deep learning" workflow

The basic steps

2. Collect a dataset

- Generate random x values between 0 and 2π
- Calculate y = sin(x)
- Plot



8

hogeschool vives

8

The "deep learning" workflow

The basic steps

2. Collect a dataset

```
# Set seed for experiment reproducibility
seed = 1
np.random.seed(seed)
tf.random.set_seed(seed)

# Number of sample datapoints
SAMPLES = 1000

# Generate a uniformly distributed set of random numbers in the range from
# 0 to 2π, which covers a complete sine wave oscillation
x_values = np.random.uniform(
    low=0, high=2*math.pi, size=SAMPLES).astype(np.float32)
```

9

hogeschool ves

9

The "deep learning" workflow

The basic steps

2. Collect a dataset

```
# Shuffle the values to guarantee they're not in order
np.random.shuffle(x_values)

# Calculate the corresponding sine values
y_values = np.sin(x_values).astype(np.float32)

# Plot our data. The 'b.' argument tells the library to print blue dots.
plt.plot(x_values, y_values, 'b.')
plt.savefig(IMG_DIR+"data.png")
```

10

hogeschool ves

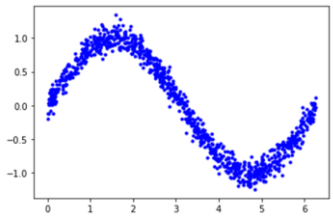
10

The "deep learning" workflow

The basic steps

Adding noise

- As the values are calculated directly from the sine function, the input data is directly giving the answer
- ML models are good at extracting underlying meaning from messy, real world data
- A small offset is added to the y values



11

hogeschool vives

11

The "deep learning" workflow

The basic steps

Adding noise

```
# Add a small random number to each y value
y_values += 0.1 * np.random.randn(*y_values.shape)

# Plot our data
plt.plot(x_values, y_values, 'b.')
plt.savefig(IMG_DIR+"data_with_noise.png")
```

12

hogeschool vives

12

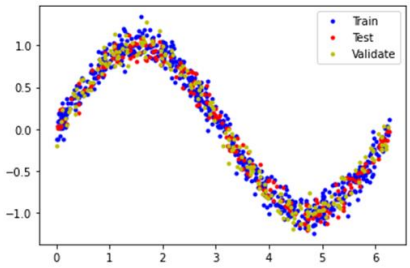
The "deep learning" workflow

The basic steps

Splitting the dataset

The data is split as follows:

- Training: 60%
- Validation: 20%
- Testing: 20%



13

hogeschool vives

13

The "deep learning" workflow

The basic steps

Splitting the dataset

```
# We'll use 60% of our data for training and 20% for testing. The remaining
20%
# will be used for validation. Calculate the indices of each section.
TRAIN_SPLIT =  int(0.6 * SAMPLES)
TEST_SPLIT = int(0.2 * SAMPLES + TRAIN_SPLIT)

# Use np.split to chop our data into three parts.
# The second argument to np.split is an array of indices where the data
will be
# split. We provide two indices, so the data will be divided into three
chunks.
x_train, x_test, x_validate = np.split(x_values, [TRAIN_SPLIT, TEST_SPLIT])
y_train, y_test, y_validate = np.split(y_values, [TRAIN_SPLIT, TEST_SPLIT])
```

14

hogeschool ves

14

The "deep learning" workflow

The basic steps

Splitting the dataset

```
# Double check that our splits add up correctly
assert (x_train.size + x_validate.size + x_test.size) ==  SAMPLES

# Plot the data in each partition in different colors:
plt.plot(x_train, y_train, 'b.', label="Train")
plt.plot(x_test, y_test, 'r.', label="Test")
plt.plot(x_validate, y_validate, 'y.', label="Validate")
plt.legend()
plt.savefig(IMG_DIR+"data_splitted.png")
```

15

hogeschool vives

15

The "deep learning" workflow

The basic steps

3. Design a model architecture

We're going to build a simple neural network model that will take an input value (in this case, x) and use it to predict a numeric output value (the sine of x). This type of problem is called a **regression**.

- First, create a **fully connected** layer of 8 neurons with the **ReLu** activation function taking a single scalar value as input
  - ReLu: Rectified Linear Unit
- Create a second layer of a single neuron to output a single value

16

hogeschool vives

16

The "deep learning" workflow

The basic steps

Compiling the model

- Compile the model using the Adam optimizer
  - Stochastic gradient descent for the optimization of a scalar parameterized objective function requiring minimization with respect to its parameters
  - https://keras.io/api/optimizers/
- Use the Mean Squared Error as loss function
  - https://keras.io/api/losses/
- Use the Mean Absolute Error as metric

17

hogeschool vives

17

The "deep learning" workflow

The basic steps

Choosing a model and compiling it

```
model_1 = tf.keras.Sequential()

# First layer takes a scalar input and feeds it through 8 "neurons". The
# neurons decide whether to activate based on the 'relu' activation
function.
model_1.add(keras.layers.Dense(8, activation='relu', input_shape=(1,)))

# Final layer is a single neuron, since we want to output a single value
model_1.add(keras.layers.Dense(1))

# Compile the model using the standard 'adam' optimizer and the mean
squared error or 'mse' loss function for regression.
model_1.compile(optimizer='adam', loss="mse", metrics=["mae"])
```

18

hogeschool vives

18

The "deep learning" workflow

The basic steps

4. Train the model

- Training data
- Epochs: how many times
- Batch size: how many samples are fed into the network before the weights and biases are updated
- Validation data

19

hogeschool vives

19

The "deep learning" workflow

The basic steps

4. Train the model

```
# Train the model
history_1 = model_1.fit(x_train, y_train, epochs=500, batch_size=64,
                        validation_data=(x_validate, y_validate))
```

20

hogeschool vives

20

The "deep learning" workflow

The basic steps

4. Train the model

```
# Draw a graph of the loss, which is the distance between
# the predicted and actual values during training and validation.
train_loss = history.history['loss']
val_loss = history.history['val_loss']
epochs = range(1, len(train_loss) + 1)
# Exclude the first few epochs so the graph is easier to read
SKIP = 100

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(epochs[SKIP:], train_loss[SKIP:], 'g.', label='Training loss')
plt.plot(epochs[SKIP:], val_loss[SKIP:], 'b.', label='Validation loss')
plt.title('Training and validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
```

21

hogeschool ves

21

The "deep learning" workflow

The basic steps

4. Train the model

```
plt.subplot(1, 2, 2)

# Draw a graph of mean absolute error, which is another way of
# measuring the amount of error in the prediction.
train_mae = history.history['mae']
val_mae = history.history['val_mae']

plt.plot(epochs[SKIP:], train_mae[SKIP:], 'g.', label='Training MAE')
plt.plot(epochs[SKIP:], val_mae[SKIP:], 'b.', label='Validation MAE')
plt.title('Training and validation mean absolute error')
plt.xlabel('Epochs')
plt.ylabel('MAE')
plt.legend()

plt.tight_layout()
plt.savefig(IMG_DIR+"mae.png")
```

22

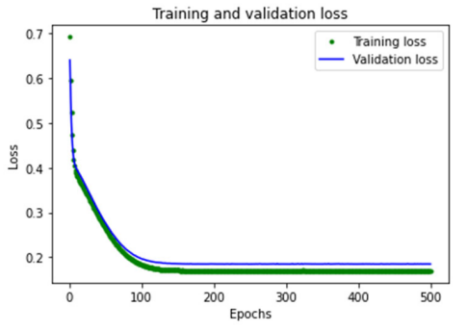
hogeschool ves

22

The "deep learning" workflow

The basic steps

Taking a look at the training metrics



23

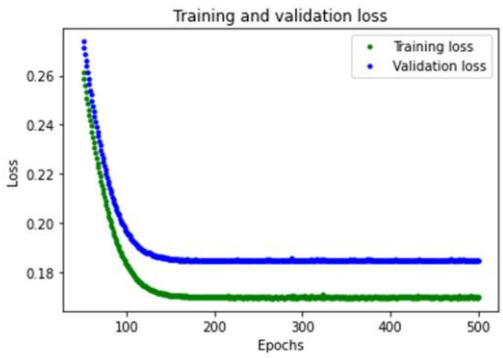
hogeschool vives

23

The "deep learning" workflow

The basic steps

Taking a look at the training metrics



24

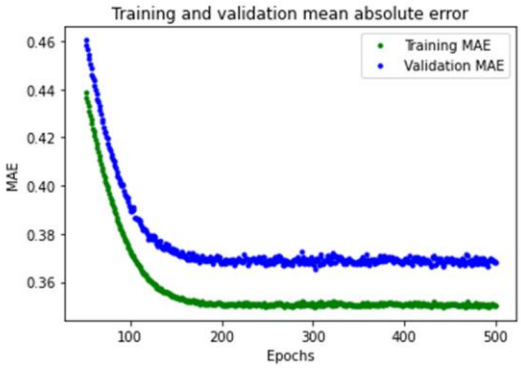
hogeschool vives

24

The "deep learning" workflow

The basic steps

Taking a look at the training metrics



25

25

The "deep learning" workflow

The basic steps

Check the model its predictions

```
# Make predictions based on our test dataset
y_test_pred = model.predict(x_test)

# Graph the predictions against the actual values
plt.clf()
plt.title('Comparison of predictions and actual values')
plt.plot(x_test, y_test, 'b.', label='Actual values')
plt.plot(x_test, y_test_pred, 'r.', label='TF predicted')
plt.legend()
plt.savefig(IMG_DIR+"data_comparison.png")
```

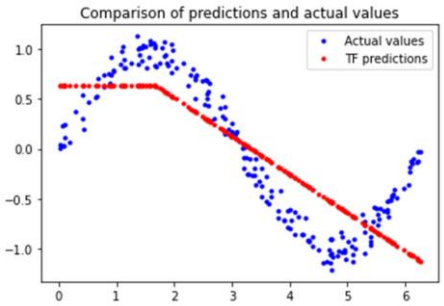
26

26

The "deep learning" workflow

The basic steps

Actual vs predicted output



Ooops!

27

hogeschool vives

27

The "deep learning" workflow

The basic steps

Design a better model!

- Make two layers of 16 neurons
- Train the model
- Check the metrics
- Evaluate the predictions

28

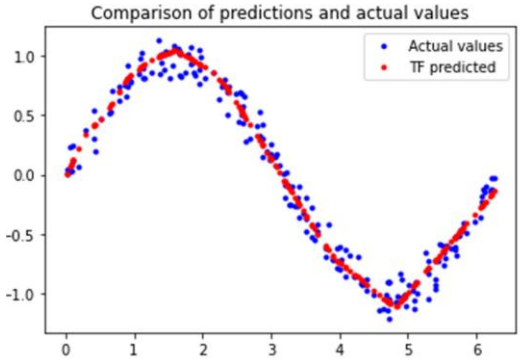
hogeschool vives

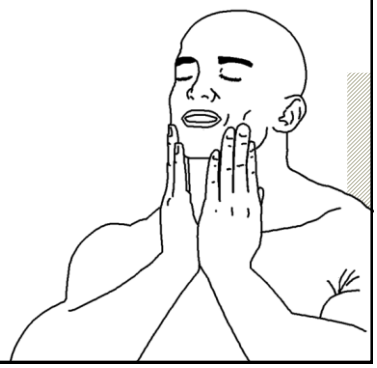
28

The "deep learning" workflow

The basic steps

Actual vs predicted output





29

29

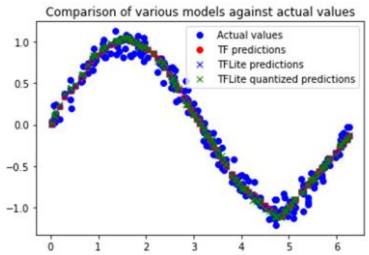
The "deep learning" workflow

The basic steps

5. Convert the model

Convert the model to TFLITE

- Without quantisation
- With quantisation



30

30

The "deep learning" workflow

The basic steps

Convert to TFLITE wo Quantization

```
# Convert the model to the TensorFlow Lite format without
quantization
converter = tf.lite.TFLiteConverter.from_saved_model(MODEL_TF)
model_no_quant_tflite = converter.convert()

# Save the model to disk
open(MODEL_NO_QUANT_TFLITE, "wb").write(model_no_quant_tflite)
```

31

31

The "deep learning" workflow

The basic steps

Convert to TFLITE w Quantization

```
# Convert the model to the TensorFlow Lite format with quantization
def representative_dataset():
  for i in range(500):
    yield([x_train[i].reshape(1, 1)])
# Set the optimization flag.
converter.optimizations = [tf.lite.Optimize.DEFAULT]
# Enforce integer only quantization (optional)
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
# Provide a representative dataset to ensure we quantize correctly.
converter.representative_dataset = representative_dataset
model_tflite = converter.convert()
```

32

32

The "deep learning" workflow

The basic steps

Save the file and convert to a hex cpp file for deployment

```
# Save the model to disk
open(MODEL_TFLITE, "wb").write(model_tflite)


import subprocess

subprocess.run("xxd -i %s > %s"%(MODEL_TFLITE, MODEL_TFLITE_MICRO),
shell=True)


# (or Format-Hex '.\model.tflite' in Windows)
# (or CertUtil -encodehex '.\model.tflite' in Windows)
```

33

33